

Practical 1

1. Create .xls file as dataset containing student records. Dataset should contain following information.

- a. Sr. No.
- b. Enrolment Number
- c. Department Name
- d. Student Name
- e. Current semester
- f. Email ID
- g. Mobile Number
- h. Current SPI
- i. CPI
- j. Professional Elective
- k. TOE Name
- l. CNS Marks
- m. SE Marks
- n. DMBI Marks
- o. WT Marks
- p. EO Marks

Create at least 20 records for the dataset and perform following operations.

- a. Use Sum function to get Grand Total of all subject marks.
- b. Find percentage with your formula for all 5 subjects.
- c. Find students is PASS or FAIL with appropriate function from percentage.
- d. Find Class with Nested if function (use marks > 75 = “distinction”, ≥ 60 = “FirstClass”, ≥ 35 = “Second Class” else “Fail”

A)

	A	B	C	D	E	F	G	H	I	J
1	Sr No	Enrolment N	Department	Student Name	Semester	Email ID	Mobile No	SPI	CPI	Professional
2	1	ENR001	CSE	Aarav Patel	5	aarav.patel@	9876543210	8.5	8.2	AI
3	2	ENR002	CSE	Diya Shah	5	diya.shah@	9876543211	9.1	8.8	ML
4	3	ENR003	IT	Rohan Meht	5	rohan.meht@	9876543212	7.9	7.8	Cloud
5	4	ENR004	IT	Ananya Verr	5	ananya.verr@	9876543213	8.3	8	Data Science
6	5	ENR005	EC	Kunal Joshi	5	kunal.joshi@	9876543214	7.5	7.6	IoT
7	6	ENR006	EC	Pooja Nair	5	pooja.nair@	9876543215	8	7.9	Embedded
8	7	ENR007	ME	Rahul Singh	5	rahul.singh@	9876543216	7.2	7.4	Robotics
9	8	ENR008	ME	Sneha Kulka	5	sneha.k@gn	9876543217	8.1	8	CAD
10	9	ENR009	CE	Mohit Agarw	5	mohit.agarw	9876543218	7.8	7.7	Construction
11	10	ENR010	CE	Neha Gupta	5	neha.gupta@	9876543219	8.4	8.2	Structural En
12	11	ENR011	CSE	Arjun Rao	6	arjun.rao@g	9876543220	9	8.7	AI
13	12	ENR012	IT	Isha Malhot	6	isha.m@gm	9876543221	8.6	8.4	Big Data
14	13	ENR013	EC	Nikhil Jain	6	nikhil.jain@	9876543222	7.6	7.8	VLSI
15	14	ENR014	ME	Kavya Iyer	6	kavya.iyer@	9876543223	8.2	8.1	Thermal
16	15	ENR015	CE	Suresh Patel	6	suresh.pate	9876543224	7.4	7.5	Transportat
17	16	ENR016	CSE	Riya Kapoor	6	riya.kapoor@	9876543225	9.3	9	Cyber Securi
18	17	ENR017	IT	Aman Khan	6	aman.khan@	9876543226	8	8.1	DevOps
19	18	ENR018	EC	Shruti Desai	6	shruti.desai	9876543227	8.5	8.3	Communica
20	19	ENR019	ME	Yash Bansal	6	yash.bansal	9876543228	7.9	7.9	Manufacturi
21	20	ENR020	CE	Priva Soni	6	priva.soni@	9876543229	8.7	8.5	Environmen

K	L	M	N	O	P	Q	R	S	T
TOE Name	CNS Marks	SE Marks	DMBI Marks	WT Marks	EO Marks	Sum	Percentage	Pass/Fail	class
TOEFL	78	82	75	80	77	392	78.4	Pass	Distinction
IELTS	85	88	84	86	83	426	85.2	Pass	Distinction
TOEFL	72	75	70	74	71	362	72.4	Pass	First class
IELTS	80	79	78	82	76	395	79	Pass	Distinction
TOEIC	68	70	72	69	71	350	70	Pass	First class
IELTS	76	78	75	77	74	380	76	Pass	Distinction
TOEFL	65	68	66	67	69	335	67	Pass	First class
IELTS	79	80	78	81	77	395	79	Pass	Distinction
TOEIC	73	75	72	74	70	364	72.8	Pass	First class
IELTS	82	83	81	84	80	410	82	Pass	Distinction
TOEFL	88	90	87	89	86	440	88	Pass	Distinction
IELTS	84	85	83	86	82	420	84	Pass	Distinction
TOEIC	71	73	74	72	70	360	72	Pass	First class
IELTS	80	81	79	82	78	400	80	Pass	Distinction
TOEFL	69	71	70	72	68	350	70	Pass	First class
IELTS	90	92	89	91	88	450	90	Pass	Distinction
TOEIC	77	78	76	79	75	385	77	Pass	Distinction
IELTS	83	85	82	84	81	415	83	Pass	Distinction
TOEFL	74	76	73	75	72	370	74	Pass	First class
IELTS	86	87	85	88	84	430	86	Pass	Distinction

Formulas:

1. $Q(\text{Sum}) = \text{SUM}(L2:P2)$
2. $R(\text{Percentage}) = (Q2/500) * 100$
3. $S(\text{Pass/Fail}) = \text{IF}(R2 \geq 33, "PASS", "FAIL")$
4. $T(\text{Class}) = \text{IF}(R2 > 75, "Distinction", \text{IF}(R2 \geq 60, "First class", \text{IF}(R2 \geq 35, "Second class", "Fail")))$

done

B)

	A	B	C	D	E	F	G	H	I	J
1	Sr No	Enrolment	Department	Student Name	Semester	Email ID	Mobile No	SPI	CPI	Profession
2	1	ENR001	CSE	Aarav Patel	5	aarav.patel	9.88E+09	8.5	8.2	AI
3	2	ENR002	CSE	Diya Shah	5	diya.shah@	9.88E+09	9.1	8.8	ML
4	3	ENR003	IT	Rohan Mehta	5	rohan.mehta	9.88E+09	7.9	7.8	Cloud
5	4	ENR004	IT	Ananya Verma	5	ananya.verma	9.88E+09	8.3	8	Data Science
6	5	ENR005	EC	Kunal Joshi	5	kunal.joshi	9.88E+09	7.5	7.6	IoT
7	6	ENR006	EC	Pooja Nair	5	pooja.nair	9.88E+09	8	7.9	Embedded
8	7	ENR007	ME	Rahul Singh	5	rahul.singh	9.88E+09	7.2	7.4	Robotics
9	8	ENR008	ME	Sneha Kulkarni	5	sneha.k@	9.88E+09	8.1	8	CAD
10	9	ENR009	CE	Mohit Aggarwal	5	mohit.aggar	9.88E+09	7.8	7.7	Constructi
11	10	ENR010	CE	Neha Gupta	5	neha.gupta	9.88E+09	8.4	8.2	Structural
12	11	ENR011	CSE	Arjun Rao	6	arjun.rao@	9.88E+09	9	8.7	AI
13	12	ENR012	IT	Isha Malhotra	6	isha.m@gr	9.88E+09	8.6	8.4	Big Data
14	13	ENR013	EC	Nikhil Jain	6	nikhil.jain@	9.88E+09	7.6	7.8	VLSI
15	14	ENR014	ME	Kavya Iyer	6	kavya.iyer	9.88E+09	8.2	8.1	Thermal
16	15	ENR015	CE	Suresh Patil	6	suresh.pat	9.88E+09	7.4	7.5	Transporta
17	16	ENR016	CSE	Riya Kapoor	6	riya.kapoo	9.88E+09	9.3	9	Cyber Secu
18	17	ENR017	IT	Aman Khanna	6	aman.khar	9.88E+09	8	8.1	DevOps
19	18	ENR018	EC	Shruti Desai	6	shruti.desa	9.88E+09	8.5	8.3	Communic
20	19	ENR019	ME	Yash Bansal	6	yash.bansa	9.88E+09	7.9	7.9	Manufactu
21	20	ENR020	CE	Priya Soni	6	priya.soni@	9.88E+09	8.7	8.5	Environme

K	L	M	N	O	P	Q	R	S
TOE Name	CNS Marks	SE Marks	DMBI Mar	WT Marks	EO Marks	Sum	Percentage	Grade
TOEFL	78	82	75	80	77	392	78.4	BB
IELTS	85	88	84	86	83	426	85.2	AB
TOEFL	72	75	70	74	71	362	72.4	BB
IELTS	80	79	78	82	76	395	79	BB
TOEIC	68	70	72	69	71	350	70	BB
IELTS	76	78	75	77	74	380	76	BB
TOEFL	65	68	66	67	69	335	67	BB
IELTS	79	80	78	81	77	395	79	BB
TOEIC	73	75	72	74	70	364	72.8	BB
IELTS	82	83	81	84	80	410	82	AB
TOEFL	88	90	87	89	86	440	88	AB
IELTS	84	85	83	86	82	420	84	AB
TOEIC	71	73	74	72	70	360	72	BB
IELTS	80	81	79	82	78	400	80	AB
TOEFL	69	71	70	72	68	350	70	BB
IELTS	90	92	89	91	88	450	90	AA
TOEIC	77	78	76	79	75	385	77	BB
IELTS	83	85	82	84	81	415	83	AB
TOEFL	74	76	73	75	72	370	74	BB
IELTS	86	87	85	88	84	430	86	AB
	1	2	0	1	0			

Formulas:

1. S (Grade)
=IF(R2>=90,"AA",IF(R2>=80,"AB",IF(R2>=60,"BB",IF(R2>40,"BC","FF"))))
2. Subject wise Distinction list =COUNTSIFS(L2:L21,".=90")

Practical 2

Create and initialize lists, tuples, sets, and dictionaries in python also perform operations like indexing, slicing, adding, removing, and sorting elements.

1.List

```
In [1]: list=[1,2,3,6,4,2,1]

In [2]: print(list)
[1, 2, 3, 6, 4, 2, 1]
```

Indexing

```
In [3]: print("element at index 0",list[0])
element at index 0 1

In [4]: print("element at index 2",list[2])
element at index 2 3
```

Slicing

```
In [5]: print("slicing the list at index 1 and 3",list[1:3])
slicing the list at index 1 and 3 [2, 3]

In [6]: print("slicing the list at til index 3",list[:3])
slicing the list at til index 3 [1, 2, 3]
```

Adding

```
In [7]: list.append(3)

In [8]: print(list)
[1, 2, 3, 6, 4, 2, 1, 3]

In [9]: list.insert(2,8)

In [10]: print(list)
[1, 2, 8, 3, 6, 4, 2, 1, 3]
```

Removing

```
In [11]: list.remove(1)

In [12]: print(list)
[2, 8, 3, 6, 4, 2, 1, 3]

In [13]: list.pop()
Out[13]: 3

In [14]: print(list)
[2, 8, 3, 6, 4, 2, 1]
```

Sorting

```
In [15]: list.sort()

In [16]: print(list)
[1, 2, 2, 3, 4, 6, 8]
```

2.Tuples

```
In [17]: tuple=(1,5,4,2,3,5)

In [18]: print(tuple)
(1, 5, 4, 2, 3, 5)
```

Indexing

```
In [19]: print("element at index 0",tuple[0])
element at index 0 1

In [20]: print("element at index 2",tuple[2])
element at index 2 4
```

Slicing

```
In [21]: print("slicing the tuple at index 1 and 3",tuple[1:3])
slicing the tuple at index 1 and 3 (5, 4)

In [22]: print("slicing the tuple at til index 3",tuple[:3])
slicing the tuple at til index 3 (1, 5, 4)
```

Sorting

```
In [23]: t=sorted(tuple)

In [24]: print(t)
[1, 2, 3, 4, 5, 5]
```

3.Set

```
In [25]: set={5,3,2,1,4,'drishi','rishika'}

In [26]: print(set)
{1, 2, 3, 4, 5, 'drishi', 'rishika'}
```

Adding

```
In [27]: set.add(6)

In [28]: print(set)
{1, 2, 3, 4, 5, 'drishi', 'rishika', 6}
```

Removing

```
In [29]: set.remove('drishi')  
  
In [30]: print(set)  
{1, 2, 3, 4, 5, 'rishika', 6}
```

4. Dictionary

```
In [31]: d={1:'pranjal',2:'Drishi','name':'rishika'}  
  
In [32]: print(d)  
{1: 'pranjal', 2: 'Drishi', 'name': 'rishika'}
```

Indexing

```
In [33]: print(d[2])  
Drishi  
  
In [34]: print(d['name'])  
rishika
```

Adding

```
In [35]: d[3]='bansi'  
  
In [36]: print(d)  
{1: 'pranjal', 2: 'Drishi', 'name': 'rishika', 3: 'bansi'}
```

Removing

```
In [37]: del d[2]  
  
In [38]: print(d)  
{1: 'pranjal', 'name': 'rishika', 3: 'bansi'}
```

Practical 3

Perform experiments using NumPy.

```
In [1]: import numpy as np
```

```
In [2]: data=np.random.randn(2,3)
```

```
In [3]: print(data)
[[-1.64635028  2.1220456   1.34951357]
 [-0.49826058  0.12070217 -1.53692233]]
```

```
In [4]: print(data*10)
[[-16.46350281  21.22045597  13.4951357 ]
 [-4.98260582   1.20702165 -15.36922335]]
```

```
In [5]: print(data+data)
[[-3.29270056  4.24409119  2.69902714]
 [-0.99652116  0.24140433 -3.07384467]]
```

```
In [6]: print(data.shape)
(2, 3)
```

```
In [7]: print(data.dtype)
float64
```

```
In [8]: data1=[6,7,3,2,1]
```

```
In [9]: a1=np.array(data1)
```

```
In [10]: print(a1)
[6 7 3 2 1]
```

```
In [11]: data2=[[2,3],[3,7]],[[3,1],[6,7]],[[2,7],[8,9]]
```

```
In [12]: a2=np.array(data2)
```

```
In [13]: print(a2)
[[[2 3]
  [3 7]]
```

```
 [[3 1]
  [6 7]]
```

```
 [[2 7]
  [8 9]]]
```

```
In [14]: print(a2.ndim)
3
```

```
In [15]: print(a2.shape)
(3, 2, 2)
```

```
In [16]: print(np.zeros(10))
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

In [17]: print(np.zeros((3,3)))
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

In [18]: print(np.empty((2,2,3)))
[[[1.03232397e-311  2.51973479e-322  0.00000000e+000]
   [0.00000000e+000  1.42419937e-306  1.16095484e-028]]

 [[8.29655075e-114  1.12959208e+277  1.46899930e+179]
  [9.08367237e+223  1.16466228e-028  1.10610224e+165]]]
```

```
In [19]: a3=np.arange(1,17).reshape(4,4)
```

```
In [20]: print(a3)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
```

```
In [21]: print(np.ones((2,4),dtype=int))
```

```
[[1 1 1 1]
 [1 1 1 1]]
```

```
In [22]: print(np.ones_like(a2))
```

```
[[[1 1]
   [1 1]]
```

```
[[[1 1]
   [1 1]]
```

```
[[[1 1]
   [1 1]]]
```

```
In [23]: print(np.full((3,4),5))
```

```
[[5 5 5 5]
 [5 5 5 5]
 [5 5 5 5]]
```

```
In [24]: print(np.eye(2))
```

```
[[1. 0.]
 [0. 1.]]
```

```
In [25]: print(np.identity(3))
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```



```
In [26]: a4=np.array([1,2,3,4,5])
```

```
In [27]: print(a4.dtype)
int64
```

```
In [28]: float_a=a4.astype(float)
```

```
In [29]: print(float_a.dtype)
float64
```

```
In [30]: a5=np.arange(1,9).reshape(2,4)
```

```
In [31]: print(a5)
[[1 2 3 4]
 [5 6 7 8]]
```

```
In [32]: print(a5*a5)
[[ 1  4  9 16]
 [25 36 49 64]]
```

```
In [33]: print(a5+a5)
[[ 2  4  6  8]
 [10 12 14 16]]
```

```
In [34]: print(a5-a5)
[[0 0 0 0]
 [0 0 0 0]]
```

```
In [35]: print(a5/2)
[[0.5 1.  1.5 2. ]
 [2.5 3.  3.5 4. ]]
```

```
In [36]: print(1/a5)
[[1.  0.5  0.33333333 0.25  ]
 [0.2  0.16666667 0.14285714 0.125 ]]
```

```
In [37]: print(a5**.5)
[[1.  1.41421356 1.73205081 2.  ]
 [2.23606798 2.44948974 2.64575131 2.82842712]]
```

```
In [38]: a6=np.arange(6)
```

```
In [39]: print(a6)
[0 1 2 3 4 5]
```

```
In [40]: a6_slice=a6[2:5]
```

```
In [41]: print(a6_slice)
[2 3 4]
```

```
In [42]: a6_slice[:]=9
```

```
In [43]: print(a6)
[0 1 9 9 9 5]
```

```
In [44]: a7=np.array([[1,2,3],[4,5,6],[7,8,9]])
```

```
In [45]: print(a7[2])  
[7 8 9]
```

```
In [46]: print(a7[1,1])  
5
```

```
In [47]: a7_slice=a7[:3]
```

```
In [48]: print(a7_slice)  
[[1 2 3]  
 [4 5 6]  
 [7 8 9]]
```

```
In [49]: a8=np.arange(1,28).reshape(3,3,3)
```

```
In [50]: print(a8)  
[[[ 1  2  3]  
   [ 4  5  6]  
   [ 7  8  9]]
```

```
 [[10 11 12]  
  [13 14 15]  
  [16 17 18]]
```

```
 [[19 20 21]  
  [22 23 24]  
  [25 26 27]]]
```

```
In [51]: print(a8[0])  
[[1 2 3]  
 [4 5 6]  
 [7 8 9]]
```

```
In [52]: print(a8[1][0])  
[10 11 12]
```

```
In [53]: print(a8[2][1][0])  
22
```

```
In [54]: a8_slice=a8[1:]
```

```
In [55]: print(a8_slice)  
[[[10 11 12]  
   [13 14 15]  
   [16 17 18]]
```

```
 [[19 20 21]  
  [22 23 24]  
  [25 26 27]]]
```

```
In [56]: a9=np.arange(1,19).reshape(3,6)
```

```
In [57]: print(a9)
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]]
```

```
In [58]: print(a9.T)
[[ 1  7 13]
 [ 2  8 14]
 [ 3  9 15]
 [ 4 10 16]
 [ 5 11 17]
 [ 6 12 18]]
```

```
In [59]: print(np.dot(a9,a9.T))
[[ 91 217 343]
 [217 559 901]
 [343 901 1459]]
```

```
In [60]: print(np.dot(a9.T,a9))
[[219 240 261 282 303 324]
 [240 264 288 312 336 360]
 [261 288 315 342 369 396]
 [282 312 342 372 402 432]
 [303 336 369 402 435 468]
 [324 360 396 432 468 504]]
```

```
In [61]: print(a9.transpose(1,0))
[[ 1  7 13]
 [ 2  8 14]
 [ 3  9 15]
 [ 4 10 16]
 [ 5 11 17]
 [ 6 12 18]]
```

```
In [62]: a10=np.arange(1,31).reshape(3,2,5)
```

```
In [63]: print(a10)
```

```
[[[ 1  2  3  4  5]
   [ 6  7  8  9 10]]
```

```
 [[11 12 13 14 15]
   [16 17 18 19 20]]
```

```
 [[21 22 23 24 25]
   [26 27 28 29 30]]]
```

```
In [64]: print(a10.transpose(0,1,2))
```

```
[[[ 1  2  3  4  5]
   [ 6  7  8  9 10]]
```

```
 [[11 12 13 14 15]
   [16 17 18 19 20]]
```

```
 [[21 22 23 24 25]
   [26 27 28 29 30]]]
```

```
In [65]: print(a10.transpose(1,2,0))
```

```
[[[ 1 11 21]
   [ 2 12 22]
   [ 3 13 23]
   [ 4 14 24]
   [ 5 15 25]]
```

```
 [[ 6 16 26]
   [ 7 17 27]
   [ 8 18 28]
   [ 9 19 29]
   [10 20 30]]]
```

```
In [66]: print(a10.transpose((2,0,1)))
```

```
[[[ 1  6]
    [11 16]
    [21 26]]
```

```
[[ 2  7]
    [12 17]
    [22 27]]
```

```
[[ 3  8]
    [13 18]
    [23 28]]
```

```
[[ 4  9]
    [14 19]
    [24 29]]
```

```
[[ 5 10]
    [15 20]
    [25 30]]]
```

```
In [67]: a11=np.arange(10)
```

```
In [68]: print(a11)
```

```
[0 1 2 3 4 5 6 7 8 9]
```

```
In [69]: print(np.sqrt(a11))
```

```
[0.         1.         1.41421356  1.73205081  2.         2.23606798
 2.44948974  2.64575131  2.82842712  3.         ]
```

```
In [70]: print(np.exp(a11))
```

```
[1.00000000e+00  2.71828183e+00  7.38905610e+00  2.00855369e+01
 5.45981500e+01  1.48413159e+02  4.03428793e+02  1.09663316e+03
 2.98095799e+03  8.10308393e+03]
```

```
In [71]: a12=np.random.randn(8)

In [72]: a13=np.random.randn(8)

In [73]: print(a12)
[ 0.32250067  1.22004351  1.90703942 -0.66374945  1.75467769  0.09633135
  0.25701273  0.04632938]

In [74]: print(a13)
[-0.16380611  0.49076678  1.34858589 -0.82443726 -1.19141875  1.0576539
 -1.47837039 -0.67162691]

In [75]: print(np.maximum(a12,a13))
[ 0.32250067  1.22004351  1.90703942 -0.66374945  1.75467769  1.0576539
  0.25701273  0.04632938]

In [76]: remainder, whole_part = np.modf(a12)

In [77]: print(remainder)
[ 0.32250067  0.22004351  0.90703942 -0.66374945  0.75467769  0.09633135
  0.25701273  0.04632938]

In [78]: print(whole_part)
[ 0.  1.  1. -0.  1.  0.  0.  0.]

In [79]: print(np.abs(a12))
[0.32250067 1.22004351 1.90703942 0.66374945 1.75467769 0.09633135
 0.25701273 0.04632938]

In [81]: print(np.sign(a12))
[ 1.  1.  1. -1.  1.  1.  1.  1.]

In [82]: print(np.ceil(a12))
[ 1.  2.  2. -0.  2.  1.  1.  1.]

In [83]: print(np.floor(a12))
[ 0.  1.  1. -1.  1.  0.  0.  0.]

In [84]: print(np.add(a12,a13))
[ 0.15869456  1.7108103   3.25562531 -1.48818672  0.56325893  1.15398526
 -1.22135765 -0.62529753]

In [85]: print(np.copysign(a12,a13))
[-0.32250067  1.22004351  1.90703942 -0.66374945 -1.75467769  0.09633135
 -0.25701273 -0.04632938]

In [86]: print(np.greater(a12,a13))
[ True  True  True  True  True False  True  True]
```

```
In [87]: print(a12>0)
[ True  True  True False  True  True  True  True]
```

```
In [88]: print(np.where(a12>0,-2,2))
[-2 -2 -2  2 -2 -2 -2 -2]
```

```
In [89]: print(np.mean(a12))
0.6175231629109541
```

```
In [90]: print(np.sum(a13))
-1.432652844880523
```

```
In [91]: a14=np.arange(15).reshape(3,5)
```

```
In [92]: print(a14.sum(axis=0))
[15 18 21 24 27]
```

```
In [93]: print(a14.mean(axis=1))
[ 2.  7. 12.]
```

```
In [94]: print(a14.cumsum(axis=0))
[[ 0  1  2  3  4]
 [ 5  7  9 11 13]
 [15 18 21 24 27]]
```

```
In [95]: print(a14.cumsum(axis=1))
[[ 0  1  3  6 10]
 [ 5 11 18 26 35]
 [10 21 33 46 60]]
```

```
In [96]: print(a14.cumprod(axis=0))
[[ 0  1  2  3  4]
 [ 0  6 14 24 36]
 [ 0 66 168 312 504]]
```

```
In [97]: print(a14.cumprod(axis=1))
[[ 0  0  0  0  0]
 [ 5 30 210 1680 15120]
 [10 110 1320 17160 240240]]
```

```
In [98]: a15=np.random.randn(50)
```

```
In [99]: print((a15>0).sum())
28
```

```
In [100]: a16=np.array([False,True,True,False,False])
```

```
In [101]: print(a16.any())
True
```

```
In [102]: print(a16.all())
False
```

```
In [103]: a17=np.random.randn(8)
```

```
In [104]: a17.sort()
```

```
In [105]: print(a17)
```

```
[-1.60740443 -1.10620888 -0.7068742  -0.09626186  0.24439625  0.68960074  
 1.77254363  2.02960366]
```

```
In [106]: name=np.array(['abc','xyz','pqr','abc','pqr'])
```

```
In [107]: print(np.unique(name))
```

```
['abc' 'pqr' 'xyz']
```

```
In [108]: print(sorted(set(name)))
```

```
[np.str_('abc'), np.str_('pqr'), np.str_('xyz')]
```

```
In [109]: ints=np.array([5,4,3,4,3,2,3,2,5,1,7])
```

```
In [110]: print(np.unique(ints))
```

```
[1 2 3 4 5 7]
```

```
In [111]: names=np.array(['abc','def','xyz','abc'])
```

```
In [112]: print(np.intersect1d(name,names))
```

```
['abc' 'xyz']
```

```
In [113]: print(np.union1d(name,names))
```

```
['abc' 'def' 'pqr' 'xyz']
```

```
In [114]: print(np.isin(name,names))
```

```
[ True  True False  True False]
```

```
In [115]: print(np.setdiff1d(name,names))
```

```
['pqr']
```

```
In [116]: print(np.setxor1d(name,names))
```

```
['def' 'pqr']
```



```
In [117]: a18=np.arange(1,17).reshape(4,4)

In [118]: print(np.diag(a18))
[ 1  6 11 16]

In [119]: print(np.trace(a18))
34

In [120]: print(np.linalg.det(a18))
-1.820448242817726e-31

In [121]: print(np.linalg.eig(a18))
EigResult(eigenvalues=array([ 3.62093727e+01, -2.20937271e+00, -2.33058886e-15,
 6.73411624e-17]), eigenvectors=array([[ -0.15115432,  0.72704996,  0.5112241 ,
 -0.07292238],
 [ -0.34923733,  0.28320876, -0.82816601, -0.30738406],
 [ -0.54732033, -0.16063243,  0.12265972,  0.83353525],
 [ -0.74540333, -0.60447363,  0.19428219, -0.45322881]]))

In [122]: print(np.linalg.inv(a18))
[[ 1.50119988e+15 -3.75299969e+14 -3.75299969e+15  2.62709978e+15]
 [-1.95155984e+16  1.95155984e+16  1.95155984e+16 -1.95155984e+16]
 [ 3.45275971e+16 -3.79052969e+16 -2.77721977e+16  3.11498974e+16]
 [-1.65131986e+16  1.87649984e+16  1.20095990e+16 -1.42613988e+16]]
```